Relatório Técnico: refatoração do núcleo financeiro da SaaS Billing Co.

Aluno: José Carlos Lee

Disciplina: Implantação e Entrega de Software (2026/2), INF/UFG

Repositório: https://github.com/jcal1998/Refatoracao

Este relatório acompanha o histórico de commits do repositório. O código legado está intacto no commit 1, e os links "L..." das tabelas apontam para as linhas exatas dele.

Sumário

1. Como executar
2. Estratégia: rede de segurança antes de refatorar
3. Diagnóstico do débito técnico (item 1)
4. Armadilhas: comportamentos que não podiam mudar
5. Refatorações aplicadas (item 2)
6. Métricas antes vs. depois
7. Mudanças de comportamento intencionais
8. Pontos em aberto para o negócio
9. Histórico de commits (item 3)

---

1. Como executar

Requisitos: JDK 21 e Maven 3.9+.

```
        mvn verify        # compila, roda os 63 testes e gera o relatório de cobertura
                          # (target/site/jacoco/index.html)
```

A suíte não usa PostgreSQL nem servidor SMTP. O workflow `.github/workflows/ci.yml` roda o mesmo comando no GitHub Actions, sem nenhum serviço auxiliar.

Em produção o serviço é montado por `BillingServiceFactory.create(System.getenv())` e precisa destas variáveis:

Variável	Uso
`BILLING_DB_URL`, `BILLING_DB_USER`, `BILLING_DB_PASSWORD`	Conexão JDBC (antes eram constantes no código)
`BILLING_EMAIL_DELIVERY_ENABLED`	`true` liga o envio real do e-mail. O padrão (`false`) reproduz o legado, que nunca enviava

Estrutura final (arquitetura hexagonal):

```
com.saas.billing
├── SubscriptionBillingService        fachada / caso de uso (orquestração)
├── domain/                           regras puras, sem I/O
│   ├── Money, BillingCommand         Value Objects
│   ├── Customer, Invoice, InvoiceStatus
│   └── PricingCalculator             motor de precificação
├── port/                             interfaces (portas)
│   ├── CustomerRepository, InvoiceRepository
│   ├── PaymentGateway, NotificationService
│   └── BillingInfrastructureException
└── infrastructure/                   adaptadores
    ├── JdbcCustomerRepository, JdbcInvoiceRepository, JdbcConnectionFactory
    ├── TokenPrefixPaymentGateway, SmtpNotificationService
    └── BillingServiceFactory         raiz de composição
```

2. Estratégia: rede de segurança antes de refatorar

A restrição era "nenhum comportamento de negócio existente pode ser quebrado". Por isso, antes de alterar qualquer linha, o commit 1 trouxe **testes de caracterização** (*golden master*). Eles registram o que o legado faz hoje, inclusive as esquisitices, e não o que ele "deveria" fazer.

Testar o legado sem infraestrutura já mostrou o tamanho da dívida. Foi preciso:

- um **driver JDBC falso** (`PostgresToH2Driver`), que intercepta a URL `jdbc:postgresql://localhost:5432/saas_db` fixa no código e a redireciona para um H2 em memória;
- um **mock estático de** `LocalDate.now()` (Mockito *inline*), porque a data não é injetável;
- observar o preço calculado **pela string de log**, o único lugar em que o legado o expõe.

Os mesmos cenários acompanharam cada commit. Nos commits 2 a 4 mudou só a forma de montar o serviço; as entradas e os valores esperados ficaram iguais (exceto as mudanças intencionais da seção 7). No commit 3 o driver falso e o Mockito foram apagados, e os testes passaram a usar portas falsas em memória e `Clock.fixed`.

No commit 4 entrou também um **teste diferencial** (`LegacyPricingEquivalenceTest`). Ele compara o `PricingCalculator` com a fórmula original em `double`, copiada literalmente, em 200 mil entradas aleatórias com semente fixa. Uma execução avulsa com 2 milhões de entradas deu:

Resultado	Cenários
Idênticos ao centavo	1.998.501 (99,925%)
Diferença de 1 centavo em valor de meio centavo exato, em que o `double` do legado ficava logo abaixo (ex.: `831.1949999999999` em vez de 831,195)	1.499 (0,075%)
Qualquer outra divergência	**0**

3. Diagnóstico do débito técnico (item 1)

Base: `SubscriptionBillingService.java` no commit 1.

#	Linha / trecho	Code smell ou princípio violado	"Juros": impacto na manutenção e na entrega contínua
1	L18-105: `processMonthlyBilling` com 88 linhas e complexidade ciclomática 18	**Long Method**; violação do **Single Responsibility Principle** (preço, consulta de cliente, pagamento, persistência, e-mail e log no mesmo método)	Qualquer ajuste numa regra de desconto obriga a reler e retestar banco, pagamento e e-mail. Gera conflitos de *merge* frequentes no mesmo arquivo e revisões lentas. Não há como testar uma regra isoladamente.
2	L18-20: 8 parâmetros soltos	**Long Parameter List**; **Flag Argument** (`isNonProfit`)	São chamadas posicionais com vários `String` vizinhos: trocar `customerId` por `paymentCardToken` compila e falha em produção. Cada novo dado de entrada quebra todos os chamadores.
3	L18, L29-57: dinheiro em `double`	**Primitive Obsession**; ausência de política de arredondamento	O erro de ponto flutuante chega ao cliente: o e-mail mostra "R$ 62.99999999999999" enquanto a fatura grava 63.00. Valores de meio centavo são perdidos (91,225 vira `91.22499999999998` e é gravado como 91.22). Não existe um lugar único para decidir como arredondar.
4	L18 `String signupDateStr` + L24 `LocalDate.parse`	**Primitive Obsession** (data como texto)	Um formato inválido só aparece em tempo de execução, dentro da rotina de cobrança, em vez de ser barrado pelo tipo na entrada.
5	L18 `String planType`	**Código morto**: parâmetro nunca lido (*Speculative Generality*)	Quem chama acredita que o plano influencia o preço, e não influencia. Induz a bugs e a "regras" inventadas por quem mantém o código.

#	Linha / trecho	Code smell ou princípio violado	"Juros": impacto na manutenção e na entrega contínua
6	L14-16: `DB_URL`, `DB_USER`, `DB_PASS = "admin123"`	**Credenciais e configuração *hardcoded*** (viola o 12-Factor, fator III)	A senha está versionada no Git. Trocar de ambiente (homologação ou produção) exige recompilar. Nenhum teste consegue apontar para outro banco.
7	L59-63, L82-87: `DriverManager` e SQL dentro do caso de uso	**Violação do Dependency Inversion Principle**; dependência direta de I/O (banco)	Sem um PostgreSQL rodando não há teste. A pipeline de CI fica impossível ou frágil: foi preciso um driver JDBC falso só para caracterizar o legado.
8	L89-96: `new SimpleEmail()` com host e porta fixos	**Violação do DIP**; dependência direta de I/O (SMTP) e de biblioteca concreta	Trocar o provedor de e-mail exige mexer na regra de cobrança. Testar o fluxo depende de um servidor SMTP.
9	L25 `LocalDate.now()` estático	**Dependência oculta do relógio** (não determinismo)	A Black Friday e a fidelidade dependem do dia em que o teste roda: um teste de novembro passa em novembro e quebra em dezembro. Congelar a data só foi possível com mock estático, mais lento e frágil.
10	L100-102: `catch (Exception e)`	**Erro mascarado** (*exception swallowing*)	Banco fora do ar vira uma linha "Erro fatal" numa lista devolvida como se a cobrança tivesse terminado. Um pagamento pode ser aprovado sem fatura gravada (falha entre L77 e L87) sem que nada alerte. O monitoramento não enxerga a falha.
11	L96 `// emailSender.send();` + L98 log "E-mail disparado"	**Código morto comentado** + **log enganoso**	O sistema afirma ter enviado recibos que nunca saíram, e ninguém sabe se o comentário é proposital ou esquecido. É uma decisão de negócio escondida num comentário.
12	L61-63, L83: `PreparedStatement`/`ResultSet` nunca fechados	**Vazamento de recurso**	Sob carga, esgota cursores e conexões do banco: a falha aparece longe da causa.
13	L75: `paymentCardToken.startsWith("tok_")`	**Gateway simulado embutido na regra** (falta de abstração)	Nenhum cartão é cobrado de verdade. Integrar um gateway real exige reescrever o serviço.
14	L32-52: `0.15`, `0.10`, `0.70`, `30.0`, `0.90`, `0.02`, `0.001`	**Magic Numbers**	Regras de negócio sem nome. Mudar um percentual exige caçar literais pelo código.

4. Armadilhas: comportamentos que não podiam mudar

O código tem detalhes que uma refatoração apressada "corrigiria" sem perceber, quebrando a regra de ouro. Todos foram preservados e cada um está travado por um teste:

Comportamento do legado	"Correção" tentadora que quebraria	Teste que trava
Meses contados por mês de calendário, **ignorando o dia** (30/09/2025 → 20/10/2026 = 13 meses)	`ChronoUnit.MONTHS.between` ou `Period`, que dariam 12 e cortariam o desconto	`monthsAreCountedByCalendarMonthIgnoringTheDay`
Limites **estritos**: `> 12` e `> 24` (12 meses não têm desconto; 24 ainda dão 10%)	`>=`	`loyaltyDiscountUsesStrictBoundaries`
`WELCOME10` vale para `monthsActive <= 1`, **inclusive meses negativos** (adesão futura)	Exigir `0 <= meses <= 1`	`welcomeCouponOnlyForUpToOneMonth`

Comportamento do legado	"Correção" tentadora que quebraria	Teste que trava
Cupom comparado com `equalsIgnoreCase` mas **sem `trim`** (`" BLACKFRIDAY "` não vale)	Normalizar com `trim().toUpperCase()`	`promoCodeIsCaseInsensitiveButNotTrimmed`
Black Friday depende do **mês corrente** (novembro), no fuso local	`Clock.systemUTC()`, que muda a data entre 21h e 0h em Brasília	`blackFridayOnlyWorksInNovember`, `todayComesFromTheInjectedClockInItsOwnZone`
Ordem das regras: fidelidade → ONG → cupom → atraso → zerar	Aplicar o cupom fixo antes do percentual de ONG (daria 49 em vez de 40)	`blackFridayIsSubtractedAfterNonProfitDiscount`, `overdueChargesApplyToTheDiscountedPrice`
Juros **simples** (0,1% × dias) sobre o mesmo preço da multa	Juros compostos, ou juros sobre preço + multa	`overdueAddsTwoPercentPenaltyPlusSimpleDailyInterest`
O preço pode ficar **negativo** no meio do cálculo e só é zerado no fim	Validar `Money >= 0` no construtor, o que lançaria exceção na Black Friday	`negativePriceIsClampedToZeroAfterOverdueCharges`, `MoneyTest.mayBecomeNegativeDuringCalculation`
Arredondar **uma única vez**, no fim	`Money` com escala 2 em toda operação: 10,05 com 13 meses e ONG daria 6,34 em vez de 6,33	`roundsOnlyOnceAtTheEnd`, `LegacyPricingEquivalenceTest`
Converter o preço base sem erro binário	`new BigDecimal(double)` (0.1 vira 0.1000000000000000055...)	`Money` só aceita `String`/`BigDecimal`; `MoneyTest.arithmeticIsExactUnlikeDouble`
Gateway = token com prefixo `tok_` (diferencia maiúsculas). Aprova até valor zero	Inventar uma integração real ou aceitar `TOK_`	`TokenPrefixPaymentGatewayTest`
Status `ACTIVE` sem diferenciar maiúsculas; status nulo = inativo	Comparar com `equals`	`statusIsComparedIgnoringCase`, `customerWithoutStatusIsTreatedAsInactive`
Pagamento recusado **ainda grava fatura `FAILED` e "envia" e-mail**	Abortar no pagamento recusado	`cardTokenWithoutTokPrefixFailsButInvoiceAndEmailStillHappen`
Cliente inexistente: **lista vazia, sem log nem fatura**	Lançar `CustomerNotFoundException`	`unknownCustomerIsSilentlyIgnored`
O e-mail **nunca é enviado** (`send()` comentado)	Descomentar o `send()`, passando a mandar e-mails a todos os clientes	`SmtpNotificationServiceTest`; o envio virou configuração explícita, desligada por padrão
`created_at` vem do `NOW()` do banco	Trocar pelo relógio da aplicação	`JdbcRepositoriesTest.savesInvoiceRoundedToCentsWithDatabaseTimestamp`
`planType` não afeta o preço	Criar regra de preço por plano	`planTypeIsIgnored` (commit 1), que justificou a remoção do parâmetro

5. Refatorações aplicadas (item 2)

A coluna "Atalho equivalente" indica a operação automatizada correspondente no IntelliJ IDEA, para referência.

Refatoração (catálogo de Fowler)	Onde	Atalho equivalente (IntelliJ)	Commit
Replace Primitive with Object	`double` → `Money` (record imutável sobre `BigDecimal`, arredondamento HALF_UP centralizado)	Refactor → Type Migration (Ctrl+Shift+F6)	2
Introduce Parameter Object	8 parâmetros → `BillingCommand` (record)	Refactor → Introduce Parameter Object	2
Change Signature / Remove Parameter	`planType` (nunca lido) removido; data passa a ser `LocalDate`	Refactor → Change Signature (Ctrl+F6)	2
Extract Interface	`CustomerRepository`, `InvoiceRepository`, `PaymentGateway`, `NotificationService`	Refactor → Extract Interface	3
Move Function / Extract Class	SQL → `Jdbc*Repository`; e-mail → `SmtpNotificationService`; regra `tok_` → `TokenPrefixPaymentGateway`	Refactor → Move (F6) / Extract Delegate	3
Introduce Parameter (injeção de dependência)	Portas e `java.time.Clock` recebidos no construtor; `LocalDate.now()` → `LocalDate.now(clock)`	Refactor → Introduce Field / Change Signature	3
Replace Error Code with Exception	`catch (Exception e)` + linha de log → `BillingInfrastructureException` propagada pelos adaptadores	—	3
Remove Dead Code	`// emailSender.send();` substituído por configuração explícita	Safe Delete (Alt+Delete)	3
Extract Class	Regras de preço → `PricingCalculator` (domínio puro)	Refactor → Extract Delegate	4
Extract Function	`applyLoyaltyDiscount`, `applyNonProfitDiscount`, `applyPromoCode`, `applyOverdueCharges`, `monthsActive`	Refactor → Extract Method (Ctrl+Alt+M)	4
Replace Magic Literal	`TWO_YEARS_LOYALTY_DISCOUNT`, `LATE_PENALTY_RATE` etc.	Refactor → Introduce Constant (Ctrl+Alt+C)	4
Consolidate Conditional Expression	O teste `promoCode != null && !trim().isEmpty()` era redundante (nulo ou em branco nunca casa com um cupom) e saiu	—	4

A fachada final, com 22 linhas e complexidade 4:

```
        public List<String> processMonthlyBilling(BillingCommand command) {
            List<String> logs = new ArrayList<>();
            Money price = pricingCalculator.calculate(command, LocalDate.now(clock));
            Optional<Customer> customer = customerRepository.findById(command.customerId());
            if (customer.isEmpty()) {
                return logs; // como no legado, cliente inexistente é ignorado sem log
            }
            if (!customer.get().isActive()) {
                logs.add("Cliente inativo. Cobrança cancelada: " + command.customerId());
                return logs;
            }

            boolean paid = paymentGateway.charge(command.paymentCardToken(), price);
            logs.add(paid ? "Pagamento processado com sucesso: " + price.format()
                    : "Falha no pagamento do cartão para: " + command.customerId());
            invoiceRepository.save(new Invoice(command.customerId(), price, InvoiceStatus.fromPayment(paid)));

            String email = customer.get().email();
            notificationService.sendInvoiceReceipt(email, price);
            logs.add("E-mail disparado para: " + email);
            return logs;
        }
```

6. Métricas antes vs. depois

Complexidade e tamanho medidos com lizard; cobertura com JaCoCo 0.8.12. "Antes" é o commit 1 (legado com os testes de caracterização) e "depois" é o commit 4.

Métrica	Antes	Depois
Linhas do método principal	88	**22**
Complexidade ciclomática do método principal	18	**4**
Parâmetros do método principal	8	**1** (`BillingCommand`)
Maior complexidade ciclomática de um método de produção	18	**5** (`applyPromoCode`)
Complexidade ciclomática média por método	18,0	**1,9**[2]
Métodos acima do limite de alerta do lizard (CCN > 15)	1	**0**
Classes/arquivos de produção	1	18
Dependências diretas de I/O no caso de uso (JDBC, SMTP, relógio estático)	3	**0**
Valores monetários em `double`	todos	**0**
`catch (Exception e)` genérico	1	**0**
Credenciais no código-fonte	1 (`admin` / `admin123`)	**0**
Código morto (linha comentada, parâmetro não lido)	2	**0**
Recursos de teste "malabaristas"	driver JDBC falso + mock estático de `LocalDate` (Mockito inline)	**nenhum** (portas falsas e `Clock.fixed`; H2 em memória só para o SQL dos adaptadores)
Testes automatizados	34[1]	**63**
Tempo da suíte (soma do surefire, mesma máquina)	2,58 s	**1,74 s** (inclui as 200 mil comparações do diferencial)
Cobertura de linhas	100% (57/57)	99,3% (139/140); **domínio + serviço: 100% (80/80)**
Cobertura de ramos	97,1% (33/34)	88,0% (44/50); **domínio + serviço: 96,9% (31/32)**

[1] O legado original não tinha nenhum teste. Os 34 foram escritos no commit 1 como rede de segurança, e só rodam graças ao driver falso e ao mock estático.

[2] O lizard não reconhece métodos declarados dentro de `record`, então os de `Money` ficaram de fora da média. Todos são triviais (complexidade 1 ou 2) e não alteram o resultado de forma relevante.

A cobertura de ramos total cai porque surgiram adaptadores de infraestrutura com caminhos que, por definição, dependem de infraestrutura real (por exemplo, o `send()` com envio ligado). No código que contém as regras de negócio, a cobertura é praticamente total.

7. Mudanças de comportamento intencionais

1. **Formato do valor**: logs e e-mail passam a mostrar sempre dois decimais. "R$ 100.0" vira "R$ 100.00", e "R$ 62.99999999999999" vira "R$ 63.00", que agora coincide com o valor da fatura.
2. **Meio centavo**: em cerca de 0,075% das combinações (seção 2), o valor exato termina em meio centavo e o `double` do legado ficava logo abaixo dele, perdendo o centavo (91,225 era gravado como 91,22). Agora arredonda uma única vez, HALF_UP, para 91,23. HALF_UP é o arredondamento comercial e o mesmo que uma coluna `NUMERIC(…,2)` do PostgreSQL aplica. A política fica numa única constante em `Money`, caso o negócio prefira HALF_EVEN.
3. **Falhas de infraestrutura** deixam de virar a linha "Erro fatal no processamento" e passam a se propagar como `BillingInfrastructureException` (exigência do item 2.4 da atividade).
4. **API**: `processMonthlyBilling` recebe um `BillingCommand`. `planType` sai porque nunca foi lido; a data de adesão chega tipada (`LocalDate`); o serviço é montado por injeção de dependências (`BillingServiceFactory`).

Todo o resto (descontos, cupons, multa, juros, status, mensagens e ordem das ações) segue idêntico, como provam os testes de caracterização e o diferencial.

8. Pontos em aberto para o negócio

Comportamentos preservados de propósito, mas que merecem uma decisão do *product owner*:

- **Envio de e-mail**: o legado nunca enviou recibos, mas registra "E-mail disparado". Para ligar, basta `BILLING_EMAIL_DELIVERY_ENABLED=true`.
- **Pagamento simulado**: nenhum cartão é cobrado de verdade (regra `tok_`). Uma integração real é só outra implementação de `PaymentGateway`.
- **Cliente inexistente** é ignorado sem log nem erro.
- **`WELCOME10` com adesão futura** (meses negativos) recebe o desconto.
- **Pagamento e fatura não são transacionais**: se a gravação da fatura falhar depois de um pagamento aprovado, agora a exceção aparece (antes era engolida), mas não há compensação automática.

9. Histórico de commits (item 3)

Commit	Hash	Conteúdo
1	`bb13b4c`	Adição do código legado base, idêntico ao original, com build Maven, CI e 34 testes de caracterização
2	`c9e178d`	Introdução dos Value Objects (`Money`, `BillingCommand`) e remoção de Primitive Obsession
3	`781e156`	Extração das interfaces de infraestrutura, injeção de dependência e de `Clock`, fim do `catch` genérico
4	`dddd09b`	Refatoração do motor de precificação (`PricingCalculator`) e validação da suíte verde (63 testes)
5	(este)	Relatório técnico

As mensagens de commit detalham cada passo e a justificativa de cada mudança.